

GUÍA DE RECOMENDACIONES PARA DISEÑO DE SOFTWARE CENTRADO EN EL USUARIO



U.N.L.P

LIC. JAVIER DIAZ - LIC. IVANA HARARI - LIC. ANA PAOLA AMADEO

GUÍA DE RECOMENDACIONES PARA DISEÑO DE SOFTWARE CENTRADO EN EL USUARIO

➡ LIC. JAVIER DIAZ

➡ LIC. IVANA HARARI

➡ LIC. ANA PAOLA AMADEO

INDICE TEMÁTICO

PRÓLOGO

INTRODUCCIÓN

CAPITULO I - FUNDAMENTOS EN HCI

¿Qué es HCI?

Evolución del HCI

Tipos de Diálogo Hombre- Computadora

Clasificación de las Interfaces del Usuario

La Independencia del Diálogo

- ¿Cómo se logra esta Estrategia?

- La aparición de un Nuevo Rol

- Una Nueva Interfaz. Diálogo Interno y Externo

- Complejidades de la Independencia de Diálogo

Componentes de la Interfaz del Usuario

Métricas de Evaluación de una Interfaz

- Objetivos de la Interfaz del Usuario

- Principios de Nielsen

Caso de Estudio

Ejercitación

Referencias

CAPITULO II - PROCESO DE DISEÑO DE LA INTERFAZ DEL USUARIO

Introducción

Características del Proceso de Desarrollo de la UI

Ciclo de Vida de la Interfaz del Usuario

Etapas de Requerimientos de la Interfaz

- Modelo SSOA

Etapas de Diseño de la Interfaz

- Características del Proceso de Diseño

La Interfaz y la Prototipación

- Prototipación: una Metodología para el Desarrollo de los Sistemas.

- Ventajas de la Prototipación

- Tipos de Prototipos

Etapas de Implementación

- Herramientas para el Desarrollo de la Interfaz

La Importancia de la Evaluación de las Interfaces del Usuario

- Métodos de Indagación

- Métodos de Inspección

Ejercitación

Referencias

CAPITULO III - NORMAS DE DISEÑO DE LAS INTERFACES VISUALES

Los Paradigmas de Interacción Visual

Menús funcionales

Múltiples ventanas

Manipulación Directa

Feedback

Dependencias

Animaciones

Cajas de Diálogo

Tablas y Browsers
Casos de Estudio
Ejercitación
Referencias

CAPITULO IV - CUALIDADES DE UNA INTERFAZ ICÓNICA

Características del diseño icónico
Ventajas del diseño icónico
Problemáticas
Formalización del diseño icónico
Casos de Estudio
Ejercitación
Referencias

CAPITULO V - LA INTERFAZ WEB Y SUS PRINCIPIOS DE DISEÑO

La estructura del sitio Web
Aspectos de organización
La página principal y su diseño
Diseño general de las páginas Web
El contenido del sitio Web
Casos de Estudio
Ejercitación
Referencias

CAPITULO VI - SIGNOS DE ADAPTACIÓN EN LA INTERFAZ DEL USUARIO

Casos de Estudio
Ejercitación

CAPITULO VII - DISEÑO DE INTERFACES ACCESIBLES

Casos de Estudio
Ejercitación

CONCLUSIONES

INTRODUCCIÓN

Millones de personas utilizan hoy en día recursos informáticos tales como computadoras, redes de computadoras, Palms, smartphones, para solventar problemas de comunicación, de trabajo o por cuestiones cotidianas.

A medida que el número de usuarios se amplía cada día, incorporando personas de formaciones y culturas dispares, aumenta a la par la demanda de los mismos de adquirir sistemas de software con mayor nivel de asistencia y facilidad de uso.

Sistemas interactivos que provean un estilo de comunicación más simple, que sean cada vez más fáciles de utilizar, de instalar, de aprender, sin requerir ningún tipo de entrenamiento específico en Informática. Además, se pretende que el software brinde mecanismos de interacción más inteligentes y que se adapte al usuario en forma eficiente.

El usuario necesita ver al sistema interactivo que está utilizando en su ordenador como una herramienta capaz de ayudarlo a resolver sus problemas, a concretar sus intenciones. Pero esto no siempre es así. Es muy común que se le sume innumerables problemas más al utilizar el software, debiendo solucionar fallos o errores producidos cuyas causas no son claras, aprender cuestiones técnicas imprevistas, incurrir a ciertas artimañas para sortear obstáculos innecesarios, adivinar cómo realizar determinada tarea recurriendo a la técnica de prueba y error.

Aunque la componente funcional esté correcta y completamente desarrollada, existen un sin número de problemas de interacción que pueden empañar el acceso y productividad de la misma, provocando en el peor de los casos, la modificación general del software o hasta su final desuso.

Estos problemas de interacción residen en una componente del software denominada “Interfaz del Usuario”, que se encarga de innumerables cuestiones como la entrada y salida de la información, visualización de los datos, soporte, manejo y control del diálogo con el usuario final, entre otros.

La interfaz del usuario o componente de diálogo es una parte del software cuyo diseño afecta el nivel de productividad del sistema general, incide en el grado de satisfacción que pueda percibir el usuario, por lo tanto es determinante para la elección, utilidad y evaluación final del software.

Como la interfaz del usuario se encuentra dentro del sistema de software, su diseño e implementación está en manos del especialista informático, no puede delegarse a un diseñador visual o a cualquier otro profesional.

Es obligación del desarrollador de sistemas atender tanto cuestiones funcionales como de interacción, dando el mismo nivel de importancia a cada una de estas partes y responsabilizándose de la integración, desarrollo, control y manejo de ambas.

Es necesario entender que el desarrollo de la interfaz del usuario forma parte del proceso de la Ingeniería del Software. Como es la única parte del sistema que interactúa directamente con los usuarios, es fundamental efectuar un estudio completo del comportamiento humano.

Se debe incluir como requerimientos indispensables en la generación del software parámetros tales como ergonometría, facilidad de uso, amigabilidad, simpleza (en términos de reducir el esfuerzo mental del usuario de llevar a cabo en la máquina la tarea en mente o pensada), flexibilidad, naturalidad (en el sentido que el usuario a través de la pantalla vea reflejado o modelado su realidad).

La interfaz del usuario debe ser diseñada teniendo en cuenta principios de diseño propios que no coinciden con la componente de aplicación. Debe ser planteada bajo la consideración de factores específicos que inciden en su gestación, tales como factores humanos principalmente, perfil de los usuarios, elementos del entorno, hardware disponible.

Su diseño parte desde un profundo conocimiento del o los usuarios finales del software y su contexto, más que desde los términos computacionales y algoritmos de programación.

Debido a que el proceso de diseño está centrado en el usuario, se trabaja con un nivel de incertidumbre y ambigüedad elevados, por lo que hace que el ciclo de vida de la interfaz sea especial y deba ser tratado en forma independiente. Esta componente se construye con la práctica, necesita ser diseñada, evaluada varias veces hasta alcanzar los requerimientos de los usuarios. Se generan versiones prototípicas de la interfaz que son puestas a prueba aunque la misma esté incompleta y aunque aún no tenga acoplada la componente funcional.

En este manual, se va a analizar estas cuestiones y se va a desplegar una serie de normativas que hacen al buen diseño de la componente de interacción, teniendo en cuenta distintos tipos de interfaces del usuario.

Mediante el estudio, exploración y puesta en práctica de estas normas y recomendaciones de diseño, se logra mejorar significativamente los canales de comunicación e interacción con el usuario final, optimizando la calidad del sistema en general.

CAPITULO I



FUNDAMENTOS EN HCI

La multiplicidad de aspectos, metodologías y de procesos a considerar, para el diseño de una correcta interfaz, (que sea óptima desde el punto de vista del usuario y acorde a la lógica del objetivo específico del sistema), hace necesario conocer todos los conceptos y terminologías que estén íntimamente relacionados con el área de HCI.

TEMAS A TRATAR

- 1 - ¿Qué es HCI?
- 2 - Evolución del HCI
- 3 - Tipos de Diálogo hombre-computadora
- 4 - Clasificación de las Interfaces del Usuario
- 5 - La Independencia del Diálogo
 - ¿Cómo se logra esta estrategia?
 - La aparición de un nuevo rol
 - Una nueva interfaz. Diálogo interno y externo
 - Complejidades de la independencia de Diálogo
- 6 - Componentes de la Interfaz del Usuario
- 7 - Métricas de Evaluación de una Interfaz
 - Objetivos de la Interfaz del usuario
 - Principios de Nielsen
- 8 - Caso de Estudio
- 9 - Ejercitación
- 10 - Referencias

I - ¿QUÉ ES HCI?

La interacción hombre-computadora, comúnmente referida con la abreviatura HCI, es el intercambio observable de información, datos y acciones entre un humano y la computadora, y viceversa.

Está constituido por el diálogo, la conversación, la comunicación que pueda fluir en ambas direcciones, entre una persona y el sistema de software que está utilizando.

La interfaz del usuario, en cambio es el medio por el cual la interacción hombre-máquina es establecida, manejada y controlada. Conformar la parte del software y hardware que permite, que ese intercambio de información y sus distintas secuencias o hilos de diálogo, se produzcan.

Estos dos términos, interacción hombre-máquina e interfaz del usuario, están muy entrelazados en el proceso de desarrollo y en estos casos se los usa como sinónimos. En muchos contextos ambos conceptos se refieren a las entradas del usuario final, su procesamiento localizado de las mismas y la presentación de las salidas o respuestas.

La transformación funcional o algorítmica de las entradas de la información a las salidas del sistema, pertenece a la componente computacional o de aplicación. Entonces, a través del HCI, se despega la lógica de la interacción con por ejemplo, la lógica del negocio.

Desde una concepción más amplia, se puede definir al HCI también como una disciplina. Se la considera un área dentro de las Ciencias de Computación que se encarga del diseño, evaluación e implementación de sistemas de computación interactivos, para el uso humano, incluyendo el estudio de todos los fenómenos concernientes a ello.

Analizando más en profundidad esta última definición, los alcances de esta nueva disciplina son muy difusos y, pueden influir en ella factores muy heterogéneos, desde aspectos tecnológicos vinculados al hardware a aspectos sociales o psicológicos del usuario. Inciden además, cuestiones diversas como facilidad de uso, productividad, eficacia para expresar distintas posibilidades, capacidad de adaptación de acuerdo a distintos usuarios, desde niños a personas de mayor edad, pasando por distintas culturas y características sociales.

Como es un área que se centra fundamentalmente en la interacción entre humanos y máquinas, se puede plantear muchas situaciones diferentes. Cuando se refiere a “humano”, puede considerarse desde un usuario particular, un grupo de usuarios, una organización o corporación, o hasta el mundo entero, como es el caso de las aplicaciones para Internet. Cuando se refiere a “máquina”, puede incluirse desde una simple PC, una workstation, una Intranet o hasta una supercomputadora con máquinas computacionales embebidas.

Además, el HCI se convierte en un área interdisciplinaria, que puede nutrirse de nociones provenientes de la Psicología, por la aplicación de teorías de procesos cognitivos y el análisis empírico del comportamiento del usuario; de la Sociología y Antropología, por el estudio de la interrelación entre la tecnología, el trabajo y las organizaciones; del Diseño Industrial, por el estudio y diseño de productos de interacción; del Diseño Visual, por el uso de las formas, colores y otros paradigmas visuales y sus diferentes aplicaciones; entre otros.

Otro punto que no hay que olvidar, son los aspectos relacionados con la Tecnología e Ingeniería, vinculados a la capacidad gráfica de las pantallas y dispositivos, para que el usuario ingrese información, ya sea por teclado, ratón, lápiz óptico, pantalla digital, voz, teléfono, entre otros medios.

Entonces, mientras exista el diálogo entre una máquina y un humano, surgirán innumerables factores relacionados con el HCI, que incluirán todos los aspectos para el diseño y construcción de interfaces del usuario adecuadas.

A continuación, se mostrará una figura que resume las interrelaciones entre los diferentes tópicos que aborda el área de HCI:

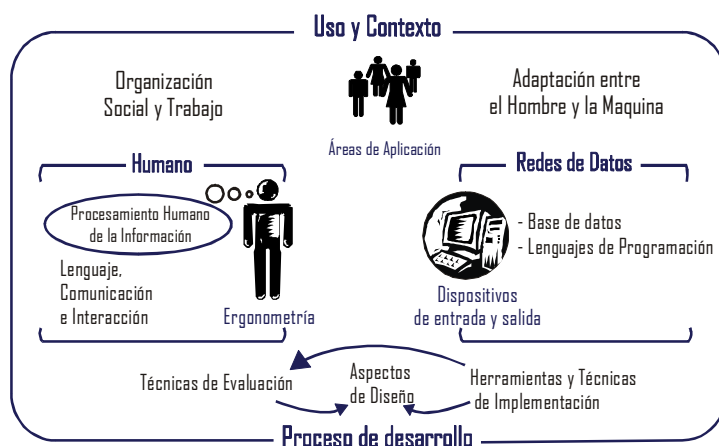


FIGURA 1.1: Variedad de Tópicos que involucra el HCI

Los sistemas de computación existen dentro de un medio social, organizacional y de trabajo. Dentro de este contexto, se encuentran las aplicaciones. Incorporar computadoras en el trabajo implica un proceso de integración.

Además del uso y contexto social de las computadoras, del lado humano debemos considerar el procesamiento humano de la información, la comunicación -el lenguaje- y las características físicas del usuario -ergonomía-.

Desde el punto de vista de la computadora, una variedad de tecnologías han sido desarrolladas para soportar la interacción con humanos, como los dispositivos de entrada y salida. Existen numerosas técnicas para organizar el diálogo y generarlo, las cuáles sirven para implementar cuestiones más específicas del mismo, como las técnicas de visualización por computadora.

Diálogos complejos o interfaces del usuario con características de avanzada, pueden llevar a consideraciones de arquitecturas de sistemas, necesarios para soportar cuestiones como tiempos de respuesta, interconectividad de redes, múltiples ventanas, interfaces multiusuarios cooperativas, utilización de bases de datos, acceso a servicios específicos, entre otros.

Finalmente, hay un proceso de desarrollo que incorpora el diseño del diálogo, herramientas de programación y técnicas de evaluación, cuyos resultados afectan las etapas anteriores, retro alimentándolas, en un proceso de continua mejoría.

II - EVOLUCIÓN DEL HCI

Debido a que las formas, los medios, los canales de interacción entre los humanos y las computadoras se modifican con el tiempo y evolucionan muy rápidamente, la disciplina de HCI es muy sensitiva a los cambios y tiempos de respuesta.

Un caso concreto fue el descubrimiento del ratón, creado por Douglas C. Engelbart en el año 1968. Este pequeño dispositivo produjo desde el punto de vista del HCI, el surgimiento de un nuevo estilo de interacción y de comunicación, incorporado en las Interfaces Gráficas. El usuario pasó, de expresar sus intenciones mediante la escritura de comandos, a demostrarlas manualmente mediante el señalamiento, el clickeo, el arrastre de objetos. Además, trajo como consecuencia toda la programación gráfica basada en el ratón y sus eventos.

Entonces, la interacción hombre-máquina o la interfaz del usuario se ve afectada por diversos factores, la misma puede variar dependiendo de diferentes situaciones, haciendo sumamente complejo el diseño de la misma. Proveer el mejor estilo de diálogo en la interfaz, la visualización y el comportamiento más óptimos no es una tarea fácil, se maneja con información incierta, con un alto nivel de ambigüedad y variabilidad temporal, que hace complicado tener parámetros absolutos o estáticos, sobre el grado de corrección de la misma.

Se requiere de un proceso de evaluación permanente y espíritu crítico para que la interacción sea más natural. Lo que se percibe como “amigable” varía con la introducción de tecnologías y el surgimiento de actitudes distintas, por ejemplo, la simplicidad y naturalidad con que los niños y adolescentes usan los mensajes de texto de la telefonía celular.



RECORDAR

❑ HCI se encuentra íntimamente relacionada con el avance de la tecnología, de las comunicaciones y de los efectos sociales que se producen con el tiempo.

Existen muchos elementos que provocan que el modo de interactuar entre un ser humano y un ordenador se modifique y evolucione con el tiempo. Entre los aspectos más influyentes en la evolución del HCI, se pueden mencionar:

ASPECTOS INFLUYENTES	EXPLICACIÓN
CUESTIONES DE HARDWARE	■ Cada vez el usuario podrá contar con hardware más potente, con mayor capacidad de memoria y velocidad. La interfaz que se diseña debe tener en cuenta aspectos de rapidez, flexibilidad.
NUEVAS TECNOLOGÍAS EN DISPOSITIVOS	■ El usuario puede conseguir dispositivos no tradicionales como medio de interacción, como ser cámaras, anteojos especiales, lápiz óptico, o como es el caso del surgimiento del teléfono celular y PALMs para acceder a aplicaciones de Internet.
TAMAÑO DE LAS PANTALLAS	■ Hay disponibles desde monitores muy grandes, finos y livianos, hasta dispositivos como los celulares con pantallas reducidas de hasta 4 a 5 líneas de texto.

	■ Los primeros, permitirán desarrollo “paper-like”, sistemas de interacción que se basan en una pantalla parecida a un papel electrónico, mientras que los segundos, permitirán portabilidad y movilidad llevando la interfaz en el bolsillo.
INCREMENTO EN INNOVACIÓN EN TÉCNICAS DE ENTRADA	■ Hay un avance específico a los medios de entrada, con la aparición de la voz, gestos, lápiz ópticos, celulares.
INCREMENTO DE USUARIOS DE DISTINTAS FORMACIONES	■ Se suman día a día potenciales usuarios como niños, adultos mayores, personas pensionadas, gente con ciertas discapacidades. ■ La interacción de las mismas con el ordenador, presenta características muy particulares a considerar dentro de la interfaz.
INCLUSIÓN DE LA COMPUTACIÓN EN TODOS LOS AMBIENTES Y CONTEXTOS	■ Muchos elementos domésticos como microondas, portones, hasta los automóviles vienen con características de programación y el usuario se acostumbra a manejar consolas y visores muy específicos, con interfaces muy distintas entre sí.
COMUNICACIÓN EN CUALQUIER LUGAR	■ Computadoras que se comunican a través de LANs de alta velocidad, nacionalmente mediante WANs y vía computación móvil con tecnologías de infrarrojo, ultrasónico o celular. ■ Por lo tanto, servicios de datos y de computación serán accesibles en cualquier lugar y momento, alterando las formas de interacción con la aplicación.
SISTEMAS CON GRAN COMPONENTE FUNCIONAL	■ Cada vez más son requeridos servicios de asistencia y búsquedas para facilitar y ayudar el trabajo de los usuarios. ■ Los sistemas interactivos deben incluir manuales, tutoriales interactivos para un aprendizaje rápido, ayudas contextuales y procesos de inferencia, ya que la gente no tendrá tiempo de aprender en forma tradicional.
DISPONIBILIDAD MASIVA DE “COMPUTER GRAPHICS”.	■ Cada vez más se aumentará la capacidad de gráficos por computadora, procesamiento de imágenes, transformaciones de imágenes, animaciones interactivas, que serán disponibles en chips baratos integrables fácilmente en cualquier computadora.
INTERFACES PARA “GROUPWARE”.	■ Las Interfaces “Groupware” permiten a grupos de personas coordinarse, encontrarse y proyectar en forma conjunta, mediante las computadoras. ■ Las mismas generan un gran impacto en la naturaleza de organización y en la distribución de labores y responsabilidades.
MULTIMEDIA, HIPERMEDIA Y COMUNIDADES VIRTUALES	■ La utilización de múltiples medios de comunicación como texto, audio, gráfico, animaciones o video, aplicados en programas multimedia como juegos, enciclopedias, cursos de entrenamientos, hace que la interfaz del usuario deba potenciar estos recursos visuales y brindar mecanismos para operarlos. ■ También, el surgimiento de las comunidades virtuales, modifican las características de la interfaz, brindando mecanismos específicos de interacción para que grupos de individuos compartan un interés común vía correo electrónico, salas de chat, o sitios de noticias – blogs-.
“UBIQUITOUS COMPUTING”	■ La disponibilidad de contar con muchas computadoras a través de un ambiente físico, siendo invisibles al usuario, requiere de cuestiones técnicas esenciales como consumo de energía, conectividad inalámbrica, más una correcta interfaz del usuario. ■ Constituye lo que se menciona como el 3er. Movimiento en Computación -Third Wave-, donde hay n computadoras por usuario. El 1er.movimiento era n usuarios por computadora mientras que el 2do. fue 1 usuario por computadora.
ACCESO ÁGIL A BASES DE DATOS MASIVAS	■ Se requieren de sistemas de visualización y métodos de búsqueda flexibles.
EL CONCEPTO DE “USER TAILORABILITY”.	■ Usuarios comunes que tendrán la capacidad de adaptar, personalizar aplicaciones para su propio uso. Podrá generar nuevas aplicaciones basadas en su entendimiento de su propio dominio. Por lo tanto serán como autores de sus propias aplicaciones.

TABLA 1.1: Aspectos que inciden en la Evolución del HCI.

En el año 1986, E.L.Hutchins, estableció que existen al menos dos metáforas o formas en que los humanos interactúan con una computadora: la “metáfora conversacional” y la del “modelo del mundo”. Estas dos metáforas corresponden unívocamente a dos tipos de diálogo, el secuencial y el asincrónico respectivamente.

En la metáfora conversacional, la interfaz del usuario se basa en representaciones implícitas de los objetos. El usuario debe describir acciones para manipular objetos en forma indirecta. Además, él debe pasar de una etapa de diálogo a la siguiente de una manera predecible y preestablecida por el sistema.

Esta metáfora, permite tanto a los desarrolladores de software como a los usuarios, visualizar una secuencia lógica y específica del comportamiento. Por tal motivo, el diálogo que genera se denomina “Diálogo Secuencial”. Este tipo de diálogo está incluido en la interacción pregunta-respuesta (request-response), lenguajes de comandos, navegación a través de menús y en las entradas de datos.

El diálogo secuencial, está relacionado con la interacción vía el teclado, que generalmente se utiliza en los lenguajes tradicionales donde las sentencias de entrada y salida, están intercaladas con el cómputo del sistema.

En la metáfora del modelo del mundo, la interfaz permite que el usuario final realice las acciones directamente sobre representaciones de los objetos del mundo real. El usuario demuestra sus intenciones, mediante el desplazamiento y manipulación de esas representaciones visuales de los objetos. Por tal motivo, el estilo de interacción característico de esta metáfora, es la “Manipulación directa” y el ratón, es el dispositivo físico apropiado para esto. También, se lo referencia como “Diálogo Asincrónico”, pues es el usuario quien decide cuándo iniciar el diálogo y con qué objeto hacerlo.

A diferencia del diálogo secuencial, donde se presenta ante el usuario una tarea por vez, aunque sea elegir entre varias opciones de un menú, en el diálogo asincrónico se dispone de muchas opciones al mismo tiempo. Es asincrónico desde el punto de vista que la secuenciación de cada camino es independiente de los demás. También al diálogo asincrónico se lo denomina “Diálogo basado en Eventos”, pues son las acciones del usuario que inician la secuencia del diálogos vistas como eventos de entrada.

Asociados a esta metáfora del modelo del mundo, se pueden encontrar los “Diálogos no lineales” o “multi-thread”, que se refieren a la multiplicidad de caminos de diálogo, de hilos de control simultáneos, de acciones alternativas disponibles para el usuario en un determinado momento. Las cajas de diálogo están relacionadas estrechamente con este tipo de diálogo.

Otro tipo de diálogo, es el “Diálogo Concurrente” que es un diálogo multi-thread, en el cuál más de un hilo de diálogo puede ser desarrollado paralelamente. Por ejemplo, un reloj que esté presente mientras que el usuario está dibujando. Además, de existir muchas alternativas de diálogo, éstas están abiertas en forma simultánea.

Entonces, se tienen:



FIGURA. 1.2: Tipos de diálogos.

En la actualidad, están surgiendo otras formas de interacción, como la “Interacción social o grupal”, donde existe un grupo de usuarios que interactúa en forma conjunta con la interfaz. Este tipo de interacción, es el provisto en las “Interfaces para Groupware”, que se explicará más adelante. Se caracteriza por tener que amoldarse a la complejidad y dinámica del grupo, y por proveer mecanismos específicos para solventar cuestiones de coordinación, comunicación y colaboración entre los miembros del mismo.

También, emergió el concepto de “Interfaz o Interacción basado en Agentes”, infundido por Pattie Maes, en donde el diálogo ya no es iniciado y controlado por el usuario como en Manipulación directa, sino que la interfaz tiene la capacidad de hacerlo. La misma puede, tanto empezar una conversación como finalizarla, puede guiar, preguntar, sugerir, enseñar, interrumpir y demás actitudes, que no son admisibles en los tipos de diálogos convencionales. Este tipo de interacción, está basado sobre una metáfora en donde la interfaz se comporta como un interlocutor humano, que tiene por objetivo colaborar con el usuario en la resolución de sus problemas. La discusión entre Interfaces basadas en Agentes versus Manipulación directa, se encuentra actualmente, en pleno auge.

Con la llegada de los sistemas de tiempo compartido en la década del 70', el dispositivo de interfaz disponible era el teclado. Esto provocaba el desarrollo de un determinado estilo de interfaz que era totalmente textual, denominado "Interfaz orientada a Comandos".

Un procesador de comandos es un sistema al cuál se le ingresa una cadena de caracteres que luego, será analizado para determinar la función apropiada de la aplicación a invocar.

Este es el caso de la interfaz del usuario más elemental, se caracteriza por ser fácil de desarrollar, se interactúa únicamente con la consola y teclado, las pantallas y las salidas del sistema son tradicionales, con impresiones a cadena de caracteres. Esto tiene ciertas limitaciones, puesto que la entrada desde el teclado provoca mayor tasa de errores, requiere que el usuario recuerde el conjunto de posibles entradas legales, y brinda una visión de la aplicación, del estilo "verbo-objeto" que no resulta amigable, ya que el usuario percibe que es la aplicación la que tiene el control total de la interacción.

Luego, a medida que se incrementó la complejidad del hardware, aparecieron las "Interfaces orientadas a Menús", que se caracteriza por presentar un conjunto de opciones, que pueden ser seleccionadas por los usuarios. Con estas interfaces se permitió abstraer la interfaz de la aplicación, pues fuerza al desarrollador a considerar el espacio de comandos y acciones como una entidad independiente.

Se caracteriza por ser fácil de usar y de implementar, la prefirieron la mayoría de los usuarios inexpertos, sin entrenamiento previo. Estudios sobre factores humanos demuestran que el proceso de lectura y elección de las opciones del menú, resulta ser más sencillo que la invocación a comandos, que requiere recordar la sintaxis de los mismos. Además, alienta a la navegación y exploración del sistema. Pero también, se le atribuyen ciertos inconvenientes como, por ejemplo, en los casos de presentar listas de opciones demasiado extensas, resulta una técnica engorrosa, o cuando hay demasiados menús anidados, puede provocar la pérdida del contexto.

Tanto las Interfaces basadas en Comandos, como las orientadas a Menús, utilizan al texto como el único medio de representación y de interacción, empleando un diálogo secuencial, por lo tanto están encuadradas en lo que se denomina "Interfaces Textuales".

A mediados de la década de los 80', debido a los grandes avances tecnológicos, la aparición de pantallas con mayor definición, dispositivos de interacción gráficos, como por ejemplo, el lápiz óptico o el ratón, comenzaron a desarrollarse para los sistemas de software, un estilo de interfaz más poderoso que el textual.

Así, surgieron las "Interfaces Gráficas" (GUI) o "Interfaces Visuales", que se caracterizaron por la utilización de recursos visuales para la representación de los objetos y por permitir la manipulación directa de los mismos, mediante un diálogo asincrónico.

Este tipo de interfaces, además de incrementar el poder representativo, también aumentó los costos y la complejidad para gerenciar este tipo de representaciones.

Como una clase específica de las interfaces visuales, se encuentran las "Interfaces Icónicas", que utilizan como medio de interacción y representación visual, exclusivamente al icono. Se puede mencionar, que el icono es una imagen, una figura, pero que tiene un significado o semántica subyacente. El mismo está determinado por una imagen que debe ser significativa y fácilmente reconocible por la comunidad de usuarios.

Tanto las "Interfaces Visuales" como las "Icónicas", serán desarrolladas en profundidad, en los Capítulos III y IV, respectivamente.

También, se puede mencionar otro tipo de interfaces que son clasificadas como "Interfaces Inteligentes", puesto que permiten que el comportamiento de la interfaz se acerque aún más al usuario, proveyendo capacidad de razonamiento, de adquisición y aplicación de conocimiento y de comunicación de ideas.

Dentro de las "Interfaces Inteligentes", se pueden citar a las "Interfaces con signos de Adaptación", "Interfaces Evolutivas" e "Interfaces con Inferencia", que presentan características disímiles respecto a sus objetivos particulares, como a su cualidad de "inteligentes".

Las "Interfaces con signos de Adaptación", brindan diferentes modos de interacción que se pueden seleccionar automáticamente de acuerdo al tipo de usuario en cuestión. Son sensibles a los perfiles individuales de los usuarios y a sus estilos de interacción.

Las "Interfaces Evolutivas", tienen la propiedad de cambiar y evolucionar con el tiempo junto con el grado de perfeccionamiento que el usuario particular va adquiriendo con el sistema. Pueden acompañar la evolución o el crecimiento que presenta el usuario ante el uso del sistema, con nuevas ayudas, mensajes más específicos, un estilo de interacción más ágil, entre otras cuestiones.

Las "Interfaces con Inferencia", tienen la capacidad de captar secuencias de acciones que el usuario repite con frecuencia. Una vez registrado esa costumbre y ante la próxima iniciativa del usuario de realizarla nuevamente, el sistema se le adelanta y brinda la posibilidad de completar la secuencia de acciones en forma automática.

La automatización de este tipo de interfaces, se llevaría a cabo mediante el reconocimiento y almacenamiento de patrones observados en el comportamiento de cada usuario. Esta adquisición de información es transparente para el usuario y conforma una base de conocimiento especializado que, junto con parámetros o criterios de evaluación y de los métodos apropiados para aplicarlo, constituyen un marco apropiado para que la interfaz provea signos de inteligencia, simplificando la labor del usuario.

También, se encuentran las interfaces presentes en los sitios de la World Wide Web, que debido al espectro inmensurable de usuarios al que está dirigida, debe tener consideraciones de diseño y desarrollo especiales. La misma va a ser explicada en detalle, en el Capítulo V.

Por último, se pueden citar otras interfaces importantes, como las “Interfaces Accesibles”, “Interfaces para Groupware”, “Interfaces puramente Conversacionales” y las “Interfaces Móviles”.

Las “Interfaces Accesibles”, son aquellas que respetan las normas del diseño universal, para que pueda ser accedida por cualquier usuario, independientemente de sus condiciones físicas o mentales. Se encuentra el Capítulo VI, en donde se va a explayar más este tema.

Las “Interfaces para Groupware”, se caracterizan por interactuar con un grupo de usuarios que tendrán objetivos comunes, recursos a compartir, un ambiente virtual de reunión. La misma deberá tener facultades para solventar la coordinación del grupo, como aspectos de colaboración y comunicación entre los miembros del mismo.

Las “Interfaces puramente Conversacionales”, se montan sobre los sistemas telefónicos y se caracterizan por no contar con una pantalla. El único medio de salida es la voz y, la entrada puede ser a través de la voz, como también, a través de los botones del dispositivo telefónico.

Las “Interfaces Móviles”, son aquellas presentes en los dispositivos inalámbricos como celulares y PALMs, que los usuarios utilizan para acceder a la aplicación. Cuestiones de interrupción, distracción, visualización de una pantalla muy reducida, el apremio por acceder a la información en ese momento y en ese lugar, hacen que la interfaz presente características de diseño muy especiales.

Vale aclarar, que un sistema interactivo puede proveer una interfaz del usuario que posea algunas características de las clases de interfaces definidas anteriormente, y las combine, generando como resultado, una interfaz híbrida.



RECORDAR

LAS INTERFACES DEL USUARIO SE PUEDEN CLASIFICAR EN

- ☐ Interfaces Textuales como:
 - las Interfaces basadas en Comandos
 - las Interfaces basadas en Menús.
- ☐ Interfaces Gráficas
- ☐ Interfaces Icónicas
- ☐ Interfaces Inteligentes como:
 - las Interfaces con Adaptación,
 - las Interfaces Evolutivas
 - con Inferencia.
- ☐ Interfaces para la Web
- ☐ Interfaces Accesibles
- ☐ Interfaces para Groupware
- ☐ Interfaces puramente Conversacionales
- ☐ Interfaces Móviles

V - INDEPENDENCIA DEL DIÁLOGO

La introducción de un sistema informático, siempre implica cambios y modificaciones a la forma de trabajo, los usuarios pueden emplear el tiempo que le resta de realizar sus actividades en forma automatizada, en otros labores, o en distintas tareas adicionales.

La interfaz de un sistema, tiene que lograr potenciar los aspectos que sean pertinentes para cada trabajo, ya sea para simplificar el tipeo, infiriendo lo que se está ingresando, para evitar la invocación errónea de un comando complejo, utilizando buenos mensajes de aclaración y confirmación, entre otras cuestiones.

El concepto es que la introducción de una solución informática, cambia la realidad y que siempre tendrá asociado modificaciones más o menos significativas, que impliquen fundamentalmente adaptar la interfaz del usuario, ya sea el “look & feel” o cambios para mejorar la productividad y eficacia de la interacción con el mismo.

Pero, al principio, esta visión del sistema interactivo no se podía aplicar. La forma tradicional de desarrollar sistemas de software, implicaba que las cuestiones de interfaz y de diálogo con el usuario, estaban fuertemente ligadas con el software computacional. Las sentencias de entrada y salida al usuario, eran tratadas sin distinción dentro del resto del código, podían estar inmersas y entremezcladas dentro de sentencias de control, causando un fuerte acoplamiento entre el flujo de diálogo con el flujo de control del sistema.

Si las cuestiones de interacción con seres humanos, se encuentran inmersas y ocultas dentro de la componente computacional del sistema, esto trae como consecuencia, una resistencia muy fuerte al desarrollo de interfaces del usuario de alta calidad. Esto se debe a que se hace complicado su identificación y tratamiento, el diálogo se convierte en una componente cada vez más resistente a cambios, no se puede profundizar en las necesidades de adaptación a cuestiones del usuario como se planteó anteriormente, y principalmente, no permite que la interfaz cumpla con un ciclo de desarrollo especial que la caracteriza, con etapas de diseño, prototipación y evaluación, que se suceden en forma iterativa, a través de un marco donde las correcciones, modificaciones y refinamientos son una constante.

En este contexto, H. Rex Hartson, Roger W. Ehrich, en el año 1981, establecieron una definición formal sobre el concepto de “Independencia de Diálogo”, en donde especificaron que un sistema interactivo, debía conformarse fundamentalmente, de dos componentes de software igualmente importantes e independientes, denominadas interfaz del usuario y componente de aplicación, o componente de diálogo y componente de cómputos respectivamente, que deben ser tratadas en forma autónoma, desde el inicio del desarrollo del sistema [Hartson, R. y Hix, D., 1989]

Esta definición, se basa en el reconocimiento de las cuestiones de interacción como una componente de software que debe ser tratada en forma especial y separada del resto del sistema, durante todo el proceso de desarrollo.

La idea de partida, es la definición de una estructura del sistema, en donde se evite la modelización conjunta e intercalada, entre la lógica de los procesos, o como se referencia actualmente como la lógica del negocio, con operaciones de E/S hacia el usuario.

En la práctica, esto significa que la componente computacional se desentiende de todo lo que abarca los aspectos de la interfaz, o sea de cómo es su estilo de interacción, cómo son las cuestiones del diálogo y los hilos posibles de conversación, si provee un lenguaje de comandos para comunicarse con el usuario final o si está basada en menús o formularios. Se despreocupa de cómo la entrada es adquirida desde el usuario y de la forma en que le son visualizados los resultados, directamente focaliza su interés en el procesamiento de la información.

La independencia del diálogo presenta principalmente dos grandes ventajas. Una, está íntimamente relacionada con la Ingeniería de Software, debido a que esta separación es crucial tanto para modificar fácilmente la interfaz en su proceso iterativo de refinamiento como para su posterior mantenimiento. Y, otra, es a nivel de factores humanos, debido a que constituye una estrategia en donde las cuestiones específicas del usuario, como sus preferencias, hábitos, costumbres, manera de ser, características de sexo o edad, experiencia particular, entre otros, pueden ser tratadas y evaluadas con énfasis y en forma especial. Esto es, permite diseñar el diálogo de una manera que la computadora se adapte a los requerimientos del usuario, convirtiéndose en una herramienta que se amolda al estilo particular del mismo. Desde esta perspectiva, el diálogo podrá conjugar la simplicidad de interacción con la optimización en el proceso de validación, proveyendo feedback y ayudas eficientes, para asistir al usuario en su trabajo y lo potencie, guiándolo en su proceso de resolución de problemas.

En este sentido, la Independencia de diálogo da lugar al tratamiento, estudio y evaluación de aspectos del usuario, suministrándole la relevancia necesaria, para poder encarar y desarrollar correctamente la componente de interfaz.



RECORDAR

□ Independencia de Diálogo es una estrategia que permite que las decisiones de diseño que afectan únicamente a la componente de diálogo, se realicen en forma independiente de aquellas que afectan a la estructura del sistema de aplicación y al corazón funcional.

¿CÓMO SE LOGRA ESTA ESTRATEGIA?

La independencia del diálogo no se implementa simplemente colocando buenos mensajes de error en uno o dos procedimientos, ni tampoco apartando todo lo relacionado con las entradas y salidas, en módulos independientes, separados del resto de las rutinas de cómputo. Esto significa una buena práctica de programación, pero no asegura la independencia del diálogo, debido a

que los procedimientos orientados al diálogo, deben estar totalmente desacoplados del código computacional y debe garantizar su modificación independientemente del resto del programa. Aún hoy, en muchos casos, el conocimiento relacionado con los detalles del diálogo siguen inmersos dentro del código de la aplicación, y la tarea de desarrollar la interfaz, continúa erróneamente ligada a una tarea de programación.

La independencia del diálogo se soporta a través de la separación del diálogo y el software de la aplicación desde las etapas iniciales del desarrollo. Esto significa que un sistema de aplicación interactivo, se compone de una componente de diálogo, a través de la cual se lleva a cabo toda la comunicación entre el usuario final y el sistema, y una componente computacional, que incluye los mecanismos de procesamiento funcional más específicos del sistema de aplicación, ámbito en donde no hay una interacción directa con el usuario final.

Más bien es que la nueva entidad, el protagonismo y la visibilidad que toma la componente de diálogo, en el desarrollo de sistemas, implica un proceso de diseño, rediseño, evaluación y mantenimiento, particular e independiente al de la componente funcional, ligándose en el momento que se requiere una rápida prototipación o ejecución del sistema total.

Como principales consecuencias de la Independencia del Diálogo, se pueden mencionar la aparición de un nuevo rol, que es la del desarrollador del diálogo, quién se va a responsabilizar de todo el proceso de desarrollo de la interfaz, y el surgimiento de un diálogo interno entre la componente de interfaz y de la aplicación, que es el que permitirá el enganche final de ambas partes, conformando el sistema interactivo en su completitud.

LA APARICIÓN DE UN NUEVO ROL: EL DESARROLLADOR DE DIÁLOGO

Por mucho tiempo, los dos principales roles involucrados en el desarrollo del software era el programador de la aplicación y el usuario final del sistema.

Estos dos roles diferentes frecuentemente tenían graves problemas en la comunicación. El programador impaciente por llegar a la codificación tenía dificultades en entender necesidades del usuario. Similarmente, el usuario muchas veces no logra comprender los costos o grado de dificultad del desarrollo de determinados requerimientos para el sistema, ni explicaciones técnicas sobre determinado comportamiento efectuado por el mismo.

La aparición del rol de analista de sistemas, proveyó un medio de entendimiento entre el programador, inmerso en un contexto completamente técnico y el del usuario del sistema.

En los últimos tiempos, ante la demanda de interfaces del usuario de alta calidad y cada vez más efectivas, la aparición de especialistas en factores humanos fueron adquiriendo una significativa importancia en los equipos de desarrollo de los sistemas de computación, puesto que focalizan su atención y estudio a las necesidades de los usuarios y a la forma de comunicación con los mismos.

Esto llevó al surgimiento de un nuevo rol, denominado desarrollador del diálogo ó también llamado autor, diseñador del diálogo, ó ingeniero de la interfaz del usuario.

El desarrollador del diálogo, es un especialista en factores humanos que se interesa y se encarga de todas las etapas del desarrollo del software de la interfaz del usuario, abarcando el diseño, la implementación y evaluación de la forma, contenido, estilo y secuenciación del diálogo. El está involucrado en todo el ciclo de vida, incluyendo análisis de tareas, especificación de requerimientos del sistema, investigación y aplicación de normas de diseño, cuestiones de factibilidad del desarrollo, dirección de todo el proceso prototípico de la interfaz.

Durante el diseño e implementación del diálogo, este desarrollador hace uso de principios psicológicos y de factores humanos para construir y, paralelamente, evaluar y refinar una interfaz que soporte una comunicación efectiva y simple, entre el ser humano y la máquina.

Debe ser muy sensitivo frente a los requerimientos cognitivos del usuario final y a sus aspectos particulares, debe profundizar por ejemplo situaciones como:

- distintas formas en que se reacciona a los distintos colores, según diferentes sociedades. Este es el caso del color rojo que en países orientales no está asociado a situaciones de riesgo;
- los diferentes hábitos de lectura, que puede afectar en el orden de recorrido de la pantalla y posicionamiento de los objetos en la misma;
- las distintas actividades que implican diferentes posturas y costumbres que condicionan el estilo de la interacción. Por ejemplo, si es mejor la entrada por voz o por teclado, en un entorno industrial o en una carga masiva de datos;
- la postura, el nivel de atención, los horarios de trabajo, que deben inducir interfaces distintas, que apunten a mejorar la productividad del usuario;
- la edad y nivel cultural del usuario, que lleva a la utilización de distintas metáforas para construir la interfaz, donde su interpretación depende de grados de madurez y aspectos sociales del mismo.

La Independencia de diálogo, define un marco en donde estas cuestiones como muchas de otras más, pueden ser profundizadas y consideradas dentro del diseño de la interfaz, permitiendo el refinamiento y revisión de las mismas, su comprobación y la realización de modificaciones rápidas, por lo que el proceso de evaluación y el ciclo de revisión se puede efectuar múltiples veces.



RECORDAR

El desarrollador del diálogo tiene el rol y la responsabilidad de:

- ☐ Efectuar los modelos necesarios sobre los usuarios, el contexto, la aplicación y aspectos tecnológicos.
- ☐ Efectuar y converger al mejor diseño de la interfaz de acuerdo a los factores humanos analizados.
- ☐ Evaluar y refinar la interfaz según su impacto directo sobre los usuarios.

UNA NUEVA INTERFAZ. DIÁLOGO INTERNO Y EXTERNO

El diálogo externo es aquel que abarca toda la interacción entre el usuario final y la componente del diálogo, es la misma interfaz hombre-computadora.

Pero, la Independencia del Diálogo ha provocado la creación de una nueva interfaz que toma lugar entre la componente del diálogo y la computacional, y en consecuencia, esto trae aparejado la aparición de un nuevo tipo de diálogo entre ambas.

La componente funcional que no tiene mecanismos propios para la comunicación directa con el usuario final, requiere de un diálogo interno necesario con la componente de diálogo.

Este diálogo, se basa principalmente en la definición concreta del intercambio de información entre ambas componentes, los tipos, formatos de datos que una espera de la otra, si van a estar ya validados o no, cuáles son las condiciones estipuladas, cuestiones de excepciones o errores que pueden surgir, quién los trata, como también en qué forma y estructura, se entregarán los resultados entre las partes. Hay que estipular claramente, cuáles serán los servicios y las funciones que una componente necesita de la otra y, las cuestiones protocolares y de sincronización necesarias.

Esta nueva interfaz interna, con su diálogo específico, son las bases para la comunicación entre el desarrollador del diálogo y el del software de la aplicación durante el proceso de diseño del sistema y es un punto crucial para llevar a cabo la ligazón del diálogo y el cómputo en los tiempos de ejecución.

El diálogo interno, y su representación formal en la etapa de diseño constituye una clave esencial para lograr la Independencia del Diálogo. Tanto la interfaz del usuario o la aplicación, pueden ser alteradas en forma independiente siempre y cuando, se mantenga la consistencia en la representación del diálogo interno común a ambas.

COMPLEJIDADES DE LA INDEPENDENCIA DE DIÁLOGO

Lamentablemente la estrategia de la Independencia de diálogo, no está exenta de inconvenientes y dificultades.

Uno de los problemas principales que se le atribuye, es que debido a que se cuenta con un nuevo rol para el desarrollo de la componente de diálogo, aumenta el grado de comunicación e interacción entre los diseñadores y los programadores, sumando problemas de ambigüedades, malas interpretaciones, deslinde de responsabilidades, pugna por toma de decisiones, entre otros inconvenientes.

Es necesario para solucionar esto, fijar claramente desde el inicio, los roles que van a intervenir, especificando puntualmente las responsabilidades de cada uno y sobre qué componente del sistema van a actuar. Como también, la definición precisa sobre el diálogo interno, estipulando cuáles son exactamente los datos, de qué tipo y formato es la información que fluirá entre la componente de interfaz y la de la aplicación, como así los servicios o funciones que se necesitan de cada una.

Otro inconveniente provocado por la Independencia del Diálogo, es que, debido a la existencia de nuevas componentes y módulos, puede causar una disminución en la performance total del sistema, a medida que aumenta la intercomunicación entre las mismas, en tiempo de ejecución. Esto lleva a la consideración de nuevas arquitecturas del sistema que enfatice y facilite por ejemplo, la ejecución concurrente entre la componente de diálogo y la computacional, y considere hardware innovador para el soporte del diálogo.

En el plano de un diálogo secuencial, la separación física entre la componente de interacción y la computacional, desde la etapa de diseño es medianamente directa. Esto se debe a que resulta fácil delinear un patrón de interacción sincrónico que considera las etapas de entrada de datos, requerida al usuario final, del proceso de cómputo de los dichos datos y, de la salida resultante provista por el sistema.

En cambio, en un diálogo con manipulación directa los costos para lograr la separación entre las dos componentes es mucho más alto, porque el usuario final realiza operaciones y acciones en la interfaz sobre las representaciones visuales de los objetos de la aplicación de una manera directa, visual y asincrónica. En este caso, la ejecución del diálogo y del cómputo tienden a estar más interrelacionados y generalmente se comparte representaciones de datos comunes de los objetos de la interfaz y de los de la aplicación subyacente.

En un diálogo con manipulación directa, hay una necesidad de acercamiento entre la interfaz y la semántica de la aplicación, por ejemplo para un simple caso de proveer feedback semántico, y esto hace más difícil la separación de las componentes requerida en la Independencia de diálogo. Esto fuerza a importantes cambios en la arquitectura de los sistemas interactivos.

De todas maneras, a menor o mayor costo, se puede lograr que las decisiones de diseño incluyendo tanto la apariencia como el comportamiento de la interfaz, puedan mantenerse independiente del software que manipula las correspondientes estructuras de datos, y en todos los casos, se comprueba que las ventajas de mantener la independencia del diálogo son mayores que sus posibles dificultades.

VI - COMPONENTES DE LA INTERFAZ DEL USUARIO

Uno de los modelos más conocidos sobre la componente de la interfaz del usuario, es el Modelo Seeheim o Seeheim Model, definido en el marco de un Workshop efectuado en Seeheim, Alemania, en el año 1982.

Este modelo, es una descripción estructural que especifica la manera en que la interfaz del usuario se relaciona con el sistema de aplicación, puntualizando la independencia entre ambas partes del sistema. Además, describe a la interfaz, mediante dos componentes: la Componente de Presentación, que conforma su parte estática, y la de Control de Diálogo, que describe la componente dinámica de la misma.

Más tarde, en 1985, el modelo fue modificado por Mark Green, perteneciente a la Universidad de Alberta, donde le introdujo una tercer componente denominada Modelo Interfaz-Aplicación, para referenciar el diálogo interno con la aplicación [Green, M., 1985].

La versión final del modelo, quedó de la siguiente manera:



FIGURA 1.3: Modelo de Seeheim, Versión modificada por M.Green

La componente de presentación es la responsable de la visualización de la interfaz, especifica las pantallas que se presentarán al usuario y el estilo de interacción. Los colores, ubicaciones de la pantalla, figuras, menús son conceptos relacionados con esta componente. En ella, se definen específicamente, los siguientes aspectos:

- Se especifica el aspecto y apariencia de la interfaz. O sea, se define la parte del “look”, del concepto de look&feel de la interfaz.
- Se define el estilo de interacción, tipos de diálogo y de interfaz.
- Se identifican los objetos de interacción.
- Se establece el esquema de ventanas.
- Se definen los dispositivos de hardware, asociados a la entrada y salida, a incorporar en la interfaz.

La componente de Control de Diálogo, está a cargo del manejo de la secuencia de eventos y del control de las técnicas de interacción provistas en la interfaz. Establece y define la parte dinámica de la misma. En esta componente, se establecen fundamentalmente, las siguientes cuestiones:

- Se especifica el comportamiento de la interfaz. O sea, la parte del “feel”, del concepto de look&feel de la interfaz.
- Se define el manejo, control, secuenciación y lógica del diálogo.
- Se controlan y se manejan los objetos de interacción presentes en la interfaz.
- Se controla y se maneja el sistema de ventanas.
- Se definen aspectos de navegación e interacción entre las diferentes ventanas o páginas que conformen el sistema.
- Se establece la estructura, secuencia, el proceso del intercambio de información entre el usuario y la aplicación.
- Se identifican, controlan y manejan los objetos computacionales necesarios a nivel de interfaz del usuario, tales como objetos sintácticos.
- Se definen los servicios propios de interfaz del usuario, o sea funciones sintácticas.
- Se controla y se administra el hardware interactivo que incluye la interfaz.
- Se especifica el tipo de asistencia, sistemas de búsqueda, tipo de feedback, que la interfaz va a proveer.
- Se especifican las cuestiones de avanzada a incorporar dentro de la interfaz, como ser aspectos de adaptación, inteligencia, colaboración que deben ser manejados y controlados a través de esta componente.

Finalmente, la componente denominada Modelo de Interfaz-Aplicación es la representación de la aplicación desde el punto de vista de la interfaz. Contiene tanto una visión de la aplicación desde la interfaz como una visión de la interfaz desde la aplicación. Incluye lo siguiente:

- Especifica el diálogo interno entre la interfaz y la aplicación.
- Permite la abstracción, por un lado, de la forma en que el usuario consulta y provee información a la aplicación y, por otro, independiza las acciones de la aplicación de las implementaciones particulares de las entradas y salidas del sistema.
- Especifica la relación entre los eventos de la interfaz y las funcionalidades de la aplicación.
- Establece la forma de comunicación entre ambas partes. Puede ser vía llamados a procedimientos o métodos, o mediante el uso de estructuras de datos comunes.

VII - MÉTRICAS DE EVALUACIÓN DE UNA INTERFAZ

Gracias a la Independencia de diálogo, filosofía que establece una separación entre la componente de interacción o interfaz del usuario con la componente de cómputos o aplicación, se puede profundizar más específicamente sobre las normas de diseño y objetivos preliminares que son propios a cada componente.

En el área especial de HCI, existen objetivos particulares que el desarrollador debe tener en cuenta, como así también principios básicos de diseño de interfaz que deben ser incluidos, si se pretende mejorar la calidad de la interacción con el sistema.

Estos aspectos no sólo deben ser considerados en el momento de diseñar una interfaz del usuario sino que además pueden ser utilizados en los procesos evaluativos de la misma durante su desarrollo, o al final, una vez que ya se encuentra en un estado productivo.

OBJETIVOS DE LA INTERFAZ DEL USUARIO

Mientras que dentro de la componente de aplicación, se consideran objetivos tales como eficiencia, portabilidad, legibilidad, entre otros aspectos, la interfaz del usuario presenta otros objetivos especiales que están íntimamente relacionados con los factores humanos más que con cuestiones técnicas o requerimientos impuestos por el sistema.

Por ser una componente que dialoga con seres humanos, se le exige cualidades como simpleza, amigabilidad, naturalidad, flexibilidad y otras características que afectan el grado de utilidad de la interfaz y por consiguiente la productividad general del resto del sistema interactivo.

La conjunción de todas las cualidades exigidas a la componente de diálogo es sintetizada con el término de “Usabilidad”.

Estos objetivos como los principios que se presentan en las próximas secciones, apuntan a mejorar la calidad de las interfaces provistas por los sistemas interactivos en pos de lograr un alto grado de satisfacción y conformidad por parte del cliente o usuario. Llevando, como consecuencia, a una segura productividad y extensa vida útil del software, finalidad que todo desarrollador de software desearía alcanzar.

Entre los principales objetivos tenemos:

a. Simplicidad:

El sistema interactivo debe ser simple de instalar, de aprender, de usar, de configurar.

Al usuario no se le puede exigir un entrenamiento sobre Computación para poder utilizar el sistema, ni que se lo someta a posteriores cursos de aprendizaje para interactuar con la interfaz.

El sistema interactivo debe proveer un diálogo natural, un ambiente amigable donde el usuario pueda encontrar toda la asistencia necesaria para resolver su problema y llevar a cabo sus intenciones.

Esto satisface a la gran demanda por parte de los usuarios de contar con sistemas de software que realmente funcionen como herramientas destinadas a simplificar el trabajo cotidiano, no a complicarlo aún más con requerimientos físicos del sistema innecesarios.

b. Confiabilidad:

Este objetivo hace referencia a que la interfaz del usuario debe ser consistente tanto en su forma de expresión como de presentación.

No puede contener diálogos ambiguos, que lleven a falsas interpretaciones o que genere dudas al usuario. Tampoco puede existir desorganización en la visualización de los datos ni heterogeneidad en los mecanismos de utilización de los servicios provistos.

El usuario debe interactuar con un ambiente integral, que presente un diseño coherente, estudiado, con un tratamiento similar en todos sus contextos y transacciones.

Esto ayuda a que los usuarios perciban una interfaz seria, cuidadosamente diseñada, en la que puedan confiar sus datos y delegar el manejo de los mismos.

c. Flexibilidad:

La interfaz del usuario debe comportarse de una manera maleable, dócil, donde el usuario pueda concretar sus intenciones en forma autónoma.

La misma debe ser tolerante a un cierto grado de error que pueda presentar el usuario y condescendiente ante equivocaciones frecuentes efectuadas por él. Puede ayudar brindando mecanismos de sugerencias o de corrección automática, más que informarle un mensaje de error.

No puede comportarse como un agente interlocutor controlador, rígido e inflexible, debe ser abierta y aceptar los tiempos y las formas que el usuario requiera.

Debe permitir que el usuario sea quien decida cuándo comenzar un diálogo y cuándo culminarlo, y quien dirija la conversación

d. Transparencia:

La componente de interfaz no puede actuar como una caja negra, donde el usuario únicamente ingrese los datos y espere los resultados.

En todo momento, la interfaz debe comunicarle al usuario sobre los detalles de la transacción y de los cómputos realizados, debe indicar el porcentaje de realización, índices del progreso e información del tiempo que falta para culminar la función que esté ejecutando.

También debe proveer mecanismos para poder interrumpir, suspender o deshacer lo efectuado por el sistema y alcanzar un estado previo deseado por el usuario.

Esto hace que la interfaz se comporte como un agente abierto, comunicador, que se manifiesta permanentemente, que informa sobre su accionar ante el usuario.

e. Ergonomía:

Este objetivo se refiere a la capacidad que presente la interfaz para amoldarse, adaptarse al estilo propio del usuario, brindando mecanismos de configuración y personalización.

La misma puede estudiar al usuario mientras está sesionando con el sistema en pos de adecuarse a su forma de interacción o también se le puede incorporar modelos computacionales con información sobre los perfiles de los usuarios, previamente investigada en las etapas preliminares del desarrollo.

De esta manera, la interfaz es percibida como un agente inteligente, que conoce al usuario, sus preferencias, sus hábitos y se acondiciona adecuadamente a él.



RECORDAR

La interfaz del usuario debe cumplir con los objetivos de:

- ☐ Simplicidad
- ☐ Confiabilidad
- ☐ Flexibilidad
- ☐ Transparencia
- ☐ Ergonomía

La suma de todas estas cualidades genera una interfaz usable, con un alto porcentaje de productividad y principalmente con un elevado grado de satisfacción por parte de los usuarios.

PRINCIPIOS DE NIELSEN

En este punto, se van a considerar los “Principios de Nielsen”, que en principio, consistió de un conjunto de diez reglas de diseño, que estipulan la forma correcta de diseñar el diálogo provisto en una interfaz del usuario, teniendo en cuenta aspectos de usabilidad.

Estos principios, considerados también como “heurísticas”, fueron definidos por Jakob Nielsen, en el año 1990, junto con la colaboración de Rolf Molich [Nielsen, J. y Mack, R., 1994].

Aunque estas normas inicialmente fueron estipuladas para interfaces textuales, sirven de base para el diseño preliminar de cualquier otro tipo de interfaz, desde la más sencilla a la más compleja.

Los principios de Nielsen, se puntualizan de la siguiente manera:

DIÁLOGO SIMPLE Y NATURAL

En este punto, se estipula la forma en que la interacción debe llevarse a cabo. El diálogo presente en los prompts, en los títulos, en los contenidos de las pantallas, en la solicitud de las entradas, como en la muestra de los resultados.

Específicamente, hay que tener en cuenta las siguientes reglas:

RECOMENDACIONES	EXPLICACIÓN
EVITAR ABUSO DE ABREVIATURAS	■ Es desprolijo que el usuario vea permanentemente sobre las pantallas del software, palabras abreviadas como “cód.”, “impr.”, “art.”, que hacen referencia respectivamente a código, imprimir y artículo.
EVITAR USO EXCESIVO DE MAYÚSCULAS	■ Hay estudios realizados en donde demostraron que el diálogo expresado en letras mayúsculas se asemeja al hablar en voz muy alta, como gritando. ■ Además de ser intimidatorio, también se le atribuye dificultad en la lectura y por consiguiente en la futura interpretación del texto. ■ Si se emplea todo el texto en mayúsculas con la intención de demostrar importancia, en realidad lo que se está provocando es intimidación y complejidad visual.
EVITAR ERRORES DE TIPO	■ El desarrollador del sistema no puede dejar plasmado en las pantallas del software, errores de ortografía, errores gramaticales ni aquellos cometidos por un mal tipeo. ■ Esto, da la idea de que el sistema no fue revisado, o que no se le dio importancia a su desarrollo.
DISTRIBUIR BIEN LA INFORMACIÓN	■ Debe planificarse el diseño general de las pantallas de un sistema. ■ Las pantallas deben tener un marco en común, con zonas bien diferenciadas para los títulos, contenidos, comandos, información, mensajes de error y demás componentes de diálogo. ■ La consistencia en el diseño visual de estos elementos, reduce la posibilidad de errores de interpretación y facilita encontrar la información.
ANALIZAR LA INFORMACIÓN RELEVANTE Y EVITAR INFORMACIÓN INNECESARIA	■ Los diálogos deberían contener únicamente la información más importante. ■ Los datos innecesarios que se encuentran en un diálogo, compiten de alguna manera con la información relevante y esto provoca la disminución de su visibilidad.
EL CONTROL DEL DIÁLOGO DEBE ESTAR DIRIGIDO POR EL USUARIO	■ Se debe evitar que el usuario en su interacción tenga que someterse a esquemas de diálogo absurdos exigidos por el sistema. ■ Por ejemplo, que el usuario tenga que “apretar dos enter y un escape” para salir de una situación determinada. ■ El usuario debe ser dueño de la interacción, debe poder iniciar el diálogo, cancelarlo, reiniciarlo según sus tiempos y necesidades.

EMPLEAR ESTILOS DE PROMPTS SIMPLES	■ Evitar prompts o preguntas del sistema que incluyan frases complejas con operadores lógicos como “o”, “y”, “no”, que dificultan la tarea interpretativa del usuario. ■ Por ejemplo, evitar la siguiente frase “<i>Salir sin Guardar</i>”
UNIFICAR EL EMPLEO DE LAS FUNCIONES PREDEFINIDAS	■ Las teclas funcionales como F1, F2,... o combinaciones de teclas, deben estar utilizadas en forma homogénea en todo el sistema y en toda circunstancia.

EMPLEAR EL LENGUAJE DEL USUARIO

Con este principio, se recomienda al diseñador de la interfaz, no trasladar su propio lenguaje, dialecto y costumbres dentro de la componente de interacción. La misma debe utilizar términos, frases o formas de diálogo, que sean familiares al ó a los usuarios del sistema.

Por lo tanto, se recomienda los siguientes puntos:

RECOMENDACIONES	EXPLICACIÓN
UTILIZAR EL LENGUAJE IDIOMA, DIALECTO Y JERGA SEGÚN LA PROFESIÓN DEL USUARIO	■ Es necesario emplear el dialecto del o los usuario/s, de acuerdo a su formación, costumbres, hábitos y preferencias.
EVITAR USO DE LENGUAJE EXTRANJERO	Palabras como “<i>exit</i>”, “<i>logout</i>”, “<i>loggueate</i>”, no deben ser empleadas.
EVITAR USO DE LENGUAJE TÉCNICO	■ Hay que evitar palabras como “<i>indexar</i>”, “<i>campo</i>”, “<i>registro</i>”, que en el marco del usuario puede tener connotaciones diferentes. ■ El diálogo debe referirse a conceptos entendidos por el usuario, más que a la implementación de la operación del sistema.
EVITAR EL USO DEL LENGUAJE REGIONAL POR PARTE DEL PROGRAMADOR	■ Expresiones como “<i>ché</i>” o “<i>vos</i>” o palabras porteñas¹, cuyo significado puede ser desconocido por algunos usuarios.
PERMITIR AL USUARIO, CONFIGURAR EL GRADO O CANTIDAD DE INFORMACIÓN QUE SE MANEJA.	■ No se debe utilizar excesivo detalle para experimentados como también escasa información para principiantes. La misma debe ser balanceada.
PERMITIR CARGAS PARCIALES DE DATOS	■ El orden de ingreso de la información en un formulario debe ser libre. ■ El usuario debería poder comenzar a completar un formulario desde cualquier punto, como también tener la posibilidad de suspenderlo, seguirlo después. ■ No debe presentar restricciones o condicionamientos sin sentido.

TABLA 1.2: Recomendaciones sobre “Emplear el Lenguaje del Usuario”

MINIMIZAR EL USO DE LA MEMORIA DEL USUARIO

En este punto, se pretende evitar que el usuario esfuerce su memoria para interactuar con el sistema.

Esto significa que el usuario no debería recurrir a carpetas manuales para recordar algún código, ni acudir a un tercero, para obtener ninguna asistencia respecto al uso del sistema. Esto

¹ Palabras porteñas: se refiere a términos y jerga utilizada en la zona del puerto de Buenos Aires, Capital Federal, Argentina.

implicaría un fracaso para el desarrollador de la interfaz.

Para evitar esto, se debe seguir las siguientes recomendaciones:

RECOMENDACIONES	EXPLICACIÓN
BRINDAR INFORMACIÓN DE CONTEXTO	■ Se debe indicar en todo contexto, en dónde se encuentra el usuario, desde dónde vino y hacia dónde puede ir. ■ Se puede además mostrar datos importantes cargados en pantallas previas, si es que el usuario debe continuar interactuando con datos relacionados.
BRINDAR INFORMACIÓN DE LA NAVEGACIÓN Y DE LA SESIÓN ACTUAL	■ Se debe mostrar información de la sesión, cuáles fueron las últimas actividades, por qué sectores estuvo trabajando. Así podrá tener presente información sobre la navegación y la interacción efectuada. ■ Saber por dónde se llegó a un determinado punto de la interacción, es importante por si se desea volver, revisar puntos anteriores o deshacer pasos. ■ El usuario puede ser interrumpido, por ejemplo por teléfono y debe contar con la información necesaria para continuar desde el mismo punto, sin perder datos ni tiempo.
VISUALIZAR LOS RANGOS DE ENTRADA ADMISIBLES POR EL SISTEMA	■ Mostrar en los campos de entrada, los posibles datos que se pueden ingresar y que ya son admisibles por el sistema, es una ayuda importante para el usuario, ya que se evitaría el tipeo de los mismos. ■ Brindar ejemplos, formatos de entrada esperados, datos por defecto no sólo ayudan al usuario a evitar errores, sino que además le enseñan cómo son los mecanismos de diálogo requeridos.
BRINDAR MECANISMOS DE ASISTENCIA Y BÚSQUEDA	■ El usuario no debería por ningún motivo, acceder a ayuda extra a la otorgada por el sistema o inclusive, utilizar medios manuales para buscar cierta información requerida por el software. ■ El sistema debe brindar todos los mecanismos de asistencia y medios de búsqueda necesarios para que el usuario pueda encontrar la información que no recuerda, en forma simple. ■ Es importante que el usuario sienta que el sistema está bajo su control y no a la inversa.

TABLA 1.4: Recomendaciones sobre "Minimizar el uso de la memoria del usuario"

CONSISTENCIA

La consistencia es un punto clave para ofrecer confiabilidad y seguridad al sistema. Se refiere a que tanto el diálogo, el aspecto visual, el aspecto terminológico, el comportamiento del sistema, se presenten ante el usuario en forma homogénea y consistente, sin ambigüedades.

Los usuarios, no deberían tener que adivinar si diferentes palabras, situaciones o acciones, significan lo mismo en distintos contextos. O, por el contrario, que un mismo término adquiriera diferentes interpretaciones, a lo largo del sistema. Todas estas situaciones hacen que la interfaz se considere inconsistente, el usuario se desconcierta y no sabe qué actitud tomar.

El poder garantizar consistencia, adquiere mayor importancia, en sistemas extensos o, cuando son desarrollados por distintas personas y en distintos momentos.

A continuación, se mostrará la tabla de recomendaciones respectivas a este punto:

RECOMENDACIONES	EXPLICACIÓN
DEBE EXISTIR UNA CONSISTENCIA TERMINOLÓGICA	■ El concepto de una función, comando, tecla o botón, debe ser siempre el mismo en todo el sistema. ■ Siempre se debe utilizar el mismo término, frase o imagen para un determinado concepto y viceversa. ■ Debe conformar una relación 1 a 1, entre la representación y el significado de cada objeto o función. ■ De esta manera se evitan falsas interpretaciones.
DEBE EXISTIR UNA CONSISTENCIA VISUAL	■ En todas las pantallas debe utilizarse los colores, espacios, distribución, tamaños, ubicaciones, sonidos y formatos en forma coherente y homogénea. ■ El usuario no debe notar grandes diferencias visuales ni tratamientos disímiles entre los contextos de un mismo sistema

TABLA 1.5: Recomendaciones sobre "Consistencia"

FEEDBACK

En este principio, se apunta a que el sistema siempre debería mantener al usuario informado de lo que está sucediendo, más que nada cuando éste manipula los objetos directamente sobre la pantalla.

El mecanismo para lograr la comunicación desde el sistema y hacia el usuario, es utilizando una técnica llamada “feedback” o retroalimentación.

Cuando se provee “feedback”, la interfaz da una respuesta gráfica o textual sobre la pantalla, frente a una acción del usuario. El feedback se refiere al efecto o reacción del sistema, que se produce por alguna operación del usuario.

El feedback, debe diseñarse de acuerdo a las siguientes consignas:

RECOMENDACIONES	EXPLICACIÓN
BRINDAR AVISOS INFORMATIVOS	■ Se debe informar al usuario, sobre qué puede realizar en un determinado momento, sobre el grado de actualización de la información que se despliega, sobre el tamaño o volumen de datos con el que va a trabajar, entre otras cosas.
BRINDAR MENSAJES ACLARATORIOS	■ La interfaz del usuario debe servir también como un medio para aclarar, explicar, detallar sobre información o comportamiento esperados del usuario.
BRINDAR MENSAJES PREVENTIVOS	■ Al usuario se le debe advertir sobre posibles situaciones indeseables que puedan ocurrir. ■ También, se le debe informar, por ejemplo, tiempos de acciones que demanden varios minutos.
DAR MENSAJES DE CONFIRMACIÓN	■ El usuario debe tener la posibilidad de confirmar cada transacción o actividad importante que haya realizado, o donde se maneje información sensible.
BRINDAR MENSAJES DE CIERRE	■ Se debe indicar cómo finalmente culminó una actividad efectuada por el usuario. ■ Se debe indicar si una transacción fue realizada exitosamente o no.
REALIZAR VALIDACIONES	■ El sistema debe validar la información ingresada por el usuario mientras él está interactuando en una manera natural. ■ No sería correcto indicar que existen datos erróneos después de un largo periodo de tiempo de interacción o cuando el usuario ya se encuentra en otro contexto o haciendo otra cosa. ■ Entonces, apenas se pueda dar retroalimentación al usuario, debe hacerse.
DAR INFORMACIÓN SOBRE LOS ESTADOS DE LOS PROCESOS	■ Se debe informar sobre cómo va el proceso que el sistema está realizando, por dónde va y cuánto falta para culminar. ■ Esto es importante en el caso de impresiones, actualizaciones, indexaciones o almacenamiento de gran volumen de información.
DAR INFORMACIÓN SOBRE EL ESTADO DEL SISTEMA Y DEL USUARIO	■ Se debe indicar el estado en que se encuentra el usuario o el sistema. Por ejemplo, si está realizando cierta labor o esperando que se corrija un error.

TABLA 1.6: Recomendaciones sobre “Feedback”

SALIDAS EVIDENTES

Este principio, se refiere a que el usuario en todo momento tenga al alcance una opción de salida de emergencia, que esté fácilmente identificable y accesible.

Esta posibilidad que le brindaría el sistema de salir del estado en que se encuentra, brinda la sensación de seguridad al usuario, pues le permitiría dejar cuanto antes alguna situación no deseada.

Las salidas deben ser siempre evidentes y se deben poder encontrar a simple vista. Las recomendaciones para ellas, son:

RECOMENDACIONES

- ☐ Brindar salidas en cada pantalla
- ☐ Brindar salidas en cada contexto
- ☐ Brindar salidas en cada operación, tarea o transacción
- ☐ Brindar salidas en cada estado
- ☐ Brindar salidas en cada momento

TABLA 1.7: Recomendaciones sobre "Salidas Evidentes"

Las salidas por lo general, se manifiestan como una opción de "Salir" propiamente dicha, pero también como opciones de "Cancelar", "Deshacer", "Reestablecer", "Suspender", "Modificar", que deben estar todas o algunas, dependiendo del caso.

Hay que considerar, que las salidas o cancelaciones, tienen gran importancia para que el usuario decida qué hacer, por ejemplo, ante la presencia de eventos externos que requieran su atención urgente.

MENSAJES DE ERROR

Eventualmente, cuando los usuarios realizan alguna combinación errónea de funciones provistas por el sistema, éste suele responder con mensajes de error.

Los mensajes de error constituyen un tipo de interacción que fluye de la máquina al usuario. Es el feedback del sistema ante la presencia de un error.

Estos mensajes de error, deben ser expresados en un lenguaje natural, su contenido debe indicar en forma clara y precisa, cuál es el problema, el lugar donde se efectuó, y sugerir alguna solución alternativa.

Hay que considerar que el usuario llegó a un punto crítico del sistema, el mismo percibe inseguridad, incertidumbre por desconocer qué hizo bien, qué hizo mal y en qué grado, el error cometido, puede afectar a los datos.

Los desarrolladores de sistemas, no deberían proveer mensajes de error en forma mecánica e impensada, sino que deben analizar cada situación de error, investigar sus posibles causas y consecuencias.

El primer cuestionamiento que el desarrollador del sistema se debe hacer, es por qué se permitió que el usuario llegue a ese punto. Quizás ante la presencia de mensajes aclaratorios, buena información o más control, el error se hubiera evitado. En principio, se debería prevenir la presencia de errores de gravedad.

Otra cuestión que se debe plantear, es que si el usuario ya se encuentra en ese punto crítico, de qué forma se lo puede ayudar para salir de él, lo más eficientemente posible.

Sobre este punto, se pueden dar las siguientes sugerencias:

RECOMENDACIONES	EXPLICACIÓN
DEBE EXISTIR MENSAJES DE ERROR	■ El sistema debe detectar situaciones de error y poder avisarlas en el debido momento y lugar.
BRINDAR INFORMACIÓN DEL ERROR	■ Como mínimo, el mensaje de error debe incluir claramente el motivo del error, explicar la causa del mismo.
EVITAR MENSAJES DE ERROR VAGOS	■ No debe mostrarse al usuario mensajes que no dicen nada en concreto o, que presenten ambigüedades. ■ Por ejemplo, el empleo de frases como "campo inválido", "número inexistente", "formato erróneo", no indican el tipo de información esperada.
DEBE DAR ALTERNATIVAS A SEGUIR	■ No sólo se debe dar a conocer cómo se llegó hasta una situación de error sino cómo salir de allí. ■ Inclusive, se debe brindar comandos para deshacer, volver a un punto anterior seguro, u ofrecer mecanismos de solución automatizados.
NO DEBE EXISTIR MENSAJES DE ERROR QUE INTIMIDEN AL USUARIO	■ No son aconsejables mensajes que emitan sonidos, o que utilicen un diálogo que alarme al usuario. ■ Tampoco se debe utilizar reprimendas, amenazas ni sanciones.

SE DEBE CATEGORIZAR LOS DIFERENTES TIPOS DE MENSAJES	■ Los mensajes de error, deben ser clasificados según sean advertencias, o si dan información sobre errores cometidos, sobre errores severos o críticos, entre otras posibilidades.
SE DEBE DETERMINAR EL GRADO DE INTERVENCIÓN DEL SISTEMA.	■ Hay que determinar claramente, a qué tipo de acción se le van a vincular los mensajes de error. Si se lo asocia a acciones elementales, compuestas o a transacciones más generales. ■ Esto afectará al grado de interrupción y de control determinado por el sistema.

TABLA 1.8: Recomendaciones sobre "Mensaje de error"

PREVENCIÓN DE ERRORES

A veces, es preferible que en vez de proveer buenos mensajes de error, se brinde un diseño más cuidadoso que prevenga problemas, para que directamente éstos no ocurran, así la persona no se encuentra en situaciones inesperadas.

En este principio, se establece la necesidad de evitar que el usuario llegue a una instancia de error. Siendo el desarrollador del sistema, se debe saber cuáles son las partes más críticas del software, con mayor posibilidad de error, y por lo tanto, se debería afianzar los mecanismos de asistencia y feedback, para que estos no se produzcan.

Generalmente, la prevención debe estar presente en la carga de datos como también, en las transacciones más relevantes y críticas, como ser eliminaciones, transferencias y modificaciones.

Algunos mecanismos de prevención son:

RECOMENDACIONES
□ Brindar rangos de entradas posibles, para que el usuario seleccione y no tipee. Mostrar ejemplos, valores por defecto, formato de entradas admisibles. □ Poner en las entradas de datos, caracteres fijos como barras "/", para evitar el tipeo de los mismos. □ Brindar mecanismos de corrección automática en el ingreso de los datos. □ Ser flexibles, aceptando una letra o, por un cero, una letra i por un 1 y demás errores generalmente cometidos por el usuario. □ Cuando se busca por un término, que el sistema busque expresiones similares, a fin de darle al usuario más información relacionada, evitándole molestias y pérdida de tiempo.

TABLA 1.8: Recomendaciones sobre "Prevención de Errores"

ATAJOS

Los atajos, son mecanismos que provee el sistema, para reducir el número de pasos necesarios para disparar ciertas acciones.

Aquí, se pide que el sistema muestre ciertos aspectos de adaptación, que considere la posibilidad de dialogar con usuarios que pueden tener distinto grado de conocimiento, formación y experiencia en el uso de aplicaciones.

Por ello, la interfaz debería proveer alternativas de manejo para que resulte cómodo y amigable, tanto para usuarios novatos como para los muy experimentados, proveyendo de esta manera mecanismos de adaptación a través de atajos o mediante la definición de macros.

Para esto, se sugiere lo siguiente:

RECOMENDACIONES
□ Brindar caminos alternativos para acelerar la interacción con el sistema. □ Brindar posibilidad de reorganizar barras de herramientas, menús, colocando sus ítems, de acuerdo a la necesidad del usuario. □ Brindar mecanismos de macros, atajos, definición de teclas de función, para acelerar el acceso a las funciones.

TABLA 1.9: Recomendaciones sobre "Atajos"

Las ayudas son componentes de asistencia muy importantes para el usuario, y esto debe ser tenido en cuenta por el desarrollador del sistema, que muchas veces no incorpora en el mismo, ningún tipo de ayudas, o las construye a la ligera sin prestar demasiada atención a las mismas.

Existe mucho análisis, sobre cómo deben ser diseñadas las ayudas de un software, más aún porque están constituidas por texto o información, que debe ser interpretada en forma rápida y eficiente.

Cuando un usuario ingresa a una ayuda, lo hace porque duda, desconoce, quiere consultar, o teme por realizar alguna acción errónea. Tiene la presión de entrar a la misma para buscar, aprender, leer, interpretar, entender sobre la problemática que le aqueja. Por lo tanto, un mal diseño de las mismas puede llegar a entorpecer, y dificultar este proceso, más que esclarecerle o asistirlo realmente.

Actualmente, los sistemas de ayuda son considerados el manual en línea del software, y tienen un objetivo explicativo, ilustrativo y de capacitación, que excede el marco de ayuda puntual, para integrarse más al sistema como un todo.

En el momento de diseñar las ayudas del sistema, se debe considerar las siguientes recomendaciones:

RECOMENDACIONES	
LAS AYUDAS DEBEN EXISTIR	■ El sistema interactivo debe ofrecer ayudas tanto a nivel del sistema en general –presente en la portada del sistema– como en cada pantalla, contexto, transacción o ante una instancia de diálogo complicada.
BRINDAR DIFERENTES TIPOS DE AYUDA	■ Debe existir tipos de ayuda apropiadas para cada caso, como ser “Tips” para ciertos botones, el botón “?” para ciertas cajas de diálogos, asistencia contextual en línea, entre otros.
DISEÑAR BIEN EL CONTENIDO DE LAS AYUDAS	■ Las ayudas deben incluir información sintáctica –propia del uso de sistema de software– como información semántica –propia del aplicativo en cuestión–
ORGANIZACIÓN DEL CONTENIDO	■ El contenido de las ayudas, debe ser organizado mediante una estructura simple, que facilite la navegación y exploración del mismo. ■ La estructura de organización de la ayuda, debe ser lógicamente planificada y representada visualmente en forma de diagramas. ■ Se deben evitar grafos hiper-conectados de páginas de información, ciclos, estructuras no balanceadas, entre otros.
PROVEER DIFERENTES FORMAS DE LECTURA	■ Proveer mecanismos de acceso a través de la navegación como de acceso directo. Permitir también una lectura secuencial.
DISEÑO GLOBAL DE LAS AYUDAS	■ Considerar la forma de organización del texto, al indentación, el desplazamiento de páginas, el tipo de letra apropiado, los tamaños de los párrafos. ■ Se debe brindar formas visuales eficientes para resaltar los puntos más importantes, el tamaño de las páginas.
BRINDAR DIFERENTES SERVICIOS DE COMUNICACIÓN ASISTENCIAL	■ Brindar diferentes mecanismos de asistencia, como búsquedas, soporte en línea, comunicación con el soporte técnico mediante el correo o fax, acceso a las FAQs o preguntas frecuentes, entre otros.

TABLA 1.10: Recomendaciones sobre “Ayudas”



RECORDAR

- Los Principios de Nielsen son normas de diseño aplicables a cualquier tipo de interfaz.
- Los mismos pueden ser utilizados también, como heurísticas de usabilidad, en un marco de evaluación de sistemas de software, ya desarrollados y en pleno uso.

CASO DE ESTUDIO 1 - FUNDAMEN-

En esta sección presentaremos un caso de estudio, donde trabajaron alumnos de la cátedra Diseño centrado en el Usuario, materia correspondiente al plan de estudios de la carrera Licenciatura en Informática de la Universidad Nacional de La Plata.

OBJETIVO

El objetivo de este trabajo fue que los alumnos tuvieran que aplicar los Principios de Nielsen en un proceso de evaluación de la interfaz del usuario de un sistema en producción.

El caso presentado tenía por objetivo pedagógico que los alumnos realicen un diagnóstico acerca de los problemas de la interfaz de un sistema, que está siendo utilizado por usuarios reales.

La evaluación se realizó en pequeños grupos y luego, entre todos los integrantes del curso, se debatieron y propusieron diferentes alternativas de solución.

El sistema que los alumnos debían analizar desde el punto de vista de la interfaz del usuario,

CARACTERÍSTICAS DEL CASO DE ESTUDIO

consiste en una administración de los cursos de formación que son organizados por la Fundación de Cardiología de la Argentina.

El mismo fue implementado por un grupo de estudiantes y presenta una serie de funciones como registro de los cursos de cardiología, altas, modificaciones y bajas de los mismos.

La pantalla inicial del sistema es la siguiente:



FIGURA -1.4

8 - DESARROLLO DEL CASO DE ESTUDIO

A continuación, se mostrará un ejemplo de un informe realizado por un grupo de estudiantes de la asignatura, en donde se evaluó el sistema en cuestión, analizando el cumplimiento de los principios de Nielsen.

Este informe está organizado de una manera en donde se hace referencia a cada principio y se demuestra partes del sistema que presentan inconvenientes en él.

DIÁLOGO SIMPLE Y NATURAL

Sobre este principio se encontraron truncamiento innecesario del box de fechas y faltas de ortografía.



FIGURA -1.5

Otro problema clásico es la salida del sistema dentro de opciones de menú que no están relacionadas. En este caso, la salida se encuentra dentro del menú "Cursos".

EMPLEAR LENGUAJE DE USUARIO

Este es un problema típico que encontramos en la mayoría de las aplicaciones, el programador utiliza lenguaje técnico para algunas operaciones. Por ejemplo



FIGURA -1.6

MINIMIZAR EL USO DE LA MEMORIA DEL USUARIO

En este punto, intento modificar la fecha de inicio de un curso. En la pantalla inicial del sistema existe un botón con la etiqueta Modificar..., pero no está activado (Figura 1.7). Presiono entonces el botón Abrir y se despliega la información del curso y un listado de personas, que no sabemos si están asociadas al curso o son las de todo el sistema. También existe el botón Modificar, pero está desactivado.(Figura 1.8)